Python Notes (Lecture 3)

Lists

1. Lists in Python

A list is a built-in data type that stores a **collection of values** in square brackets `[]`. A list can store elements of different types (integer, float, string, etc.).

```
marks = [87, 64, 33, 95, 76]
student = ["Karan", 85, "Delhi"]

print(marks)
print(student)
print(marks[0])       # first element
print(student[2])
```

Output:

```
[87, 64, 33, 95, 76]
['Karan', 85, 'Delhi']
87
Delhi
```

Like strings, list indexing starts from `0`. But unlike strings, lists are **mutable**: you can change their elements.

```
student = ["Karan", 85, "Delhi"]
student[0] = "Arjun"      # allowed in Python
print(student)
print(len(student))       # length of the list
```

Output:

```
['Arjun', 85, 'Delhi']
3
```

2. List Slicing

List slicing works the same way as string slicing.

```
list_name[starting_idx : ending_idx]    # ending_idx is NOT included
```

```
marks = [87, 64, 33, 95, 76]

print(marks[1:4])
print(marks[:4])      # same as marks[0:4]
print(marks[1:])      # same as marks[1:len(marks)]
print(marks[-3:-1])   # negative indexes
```

Output:

```
[64, 33, 95]
[87, 64, 33, 95]
[64, 33, 95, 76]
[33, 95]
```

---

3. List Methods

Methods are functions we call on a list with a dot. These methods **change the original list**.

Method	What it does
`nums.append(el)`	Adds one element at the end
`nums.sort()`	Sorts in ascending order
`nums.sort(reverse=True)`	Sorts in descending order
`nums.reverse()`	Reverses the list
`nums.insert(idx, el)`	Inserts element at the given index
`nums.remove(el)`	Removes the first occurrence of the element
`nums.pop(idx)`	Removes the element at the given index

```python
nums = [2, 1, 3]
nums.append(4)
print(nums)

nums = [2, 1, 3]
nums.sort()
print(nums)

nums.sort(reverse=True)
print(nums)

nums = [2, 1, 3]
nums.reverse()
print(nums)
```

Output:

```
[2, 1, 3, 4]
[1, 2, 3]
[3, 2, 1]
[3, 1, 2]
```

```python
nums = [2, 1, 3]
nums.insert(1, 5)     # insert 5 at index 1
print(nums)

nums = [2, 1, 3, 1]
nums.remove(1)        # removes first 1 only
print(nums)

nums = [2, 1, 3, 1]
nums.pop(2)           # removes element at index 2
print(nums)
```

Output:

```
[2, 5, 1, 3]
[2, 3, 1]
[2, 1, 1]
```

Tip: avoid naming a variable `list`, because that hides Python's built-in `list()` function.

Tuples

1. Tuples in Python

A tuple is a built-in data type that stores an **immutable** sequence of values in round brackets `( )`. Once created, its elements cannot be changed.

```
tup = (87, 64, 33, 95, 76)

print(tup[0])
print(tup[1:3])
print(len(tup))
```

Output:

```
87
(64, 33)
5
```

```
tup = (87, 64, 33, 95, 76)
tup[0] = 43        # Error!
```

Output:

```
TypeError: 'tuple' object does not support item assignment
```

Creating tuples:

```
tup1 = ()           # empty tuple
tup2 = (1,)         # single element: the comma is required
tup3 = (1, 2, 3)

print(type(tup2))
print(type((1)))    # without comma, it is just an int
```

Output:

```
<class 'tuple'>
<class 'int'>
```

2. Tuple Methods

Method	What it does
`tup.index(el)`	Returns the index of the first occurrence
`tup.count(el)`	Counts the total occurrences

```
tup = (2, 1, 3, 1)

print(tup.index(1))
print(tup.count(1))
```

Output:

```
1
2
```

List vs Tuple

List	Tuple
Written with square brackets `[ ]`	Written with round brackets `( )`
Mutable: elements can be changed	Immutable: elements cannot be changed
Many methods (append, sort, pop...)	Only 2 methods (index, count)

Let's Practice

Q1. Ask the user to enter the names of their 3 favorite movies and store them in a list.

```
movies = []

movies.append(input("Enter 1st movie: "))
movies.append(input("Enter 2nd movie: "))
movies.append(input("Enter 3rd movie: "))

print(movies)
```

Output:

```
Enter 1st movie: 3 Idiots
Enter 2nd movie: Dangal
Enter 3rd movie: Inception
['3 Idiots', 'Dangal', 'Inception']
```

Q2. Check if a list is a palindrome (reads the same forwards and backwards).

Examples: `[1, 2, 3, 2, 1]` and `[1, "abc", "abc", 1]` are palindromes. Hint: use the `copy()` method.

```
nums = [1, 2, 3, 2, 1]

copy_nums = nums.copy()   # make a copy first
copy_nums.reverse()       # reverse the copy, not the original

if nums == copy_nums:
    print("Palindrome")
else:
    print("Not a palindrome")
```

Output:

```
Palindrome
```

We use `copy()` because `reverse()` changes the list itself. Without a copy, we would lose the original order.

Q3. Count the number of students with the "A" grade in the following tuple.

```
grades = ("C", "D", "A", "A", "B", "B", "A")
print("Students with A grade:", grades.count("A"))
```

Output:

```
Students with A grade: 3
```

Q4. Store the above values in a list and sort them from "A" to "D".

```
grades = ["C", "D", "A", "A", "B", "B", "A"]
grades.sort()
print(grades)
```

Output:

```
['A', 'A', 'A', 'B', 'B', 'C', 'D']
```

APNA COLLEGE